

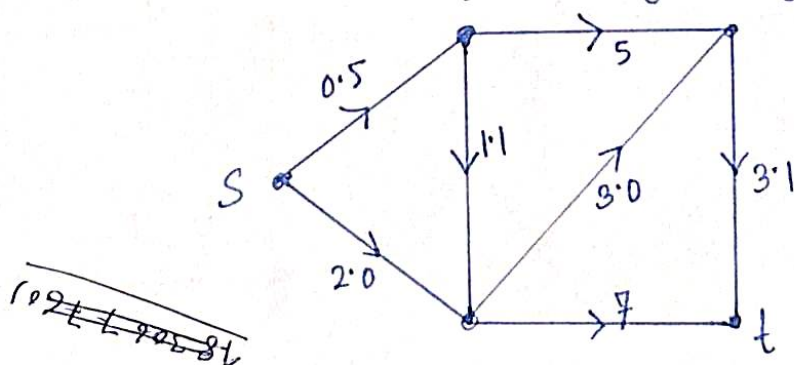
Single source shortest Path

(1)

Today we are going to talk about the single source shortest path problem. Earlier we talked about Prim's algorithm for computing the minimum spanning trees. We will find that the algorithm for computing shortest path is strikingly similar to the Prim's algorithm. So let us now see what the single source shortest path problem is and what we do in this problem does. Once again we are given a graph $G=(V,E)$ and the graph is an undirected graph. This problem is well defined for even for undirected graphs. Here we are assuming that the given graph is directed. So for the explanation let us take a graph, each edge of which has a length l . The length l is a function from the edges to non-negative reals

$$l: E \rightarrow \mathbb{R}^+$$

we can have edges of length 0 also



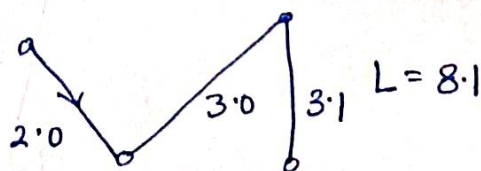
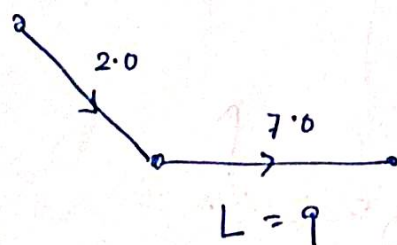
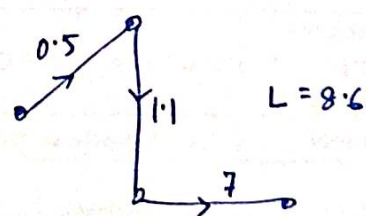
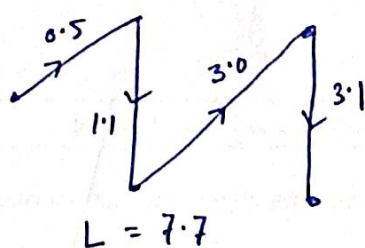
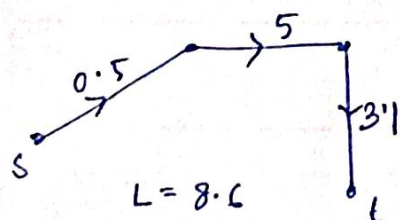
(2)

This graph could represent a network of the roads in a city let's say. Each length corresponds to the distance from one end point to another end point and there are one way roads. There are many other applications of this problem which will be discussed later.

So what the problem is? The problem is to find the shortest path from the point S (called the source) to the point t also called destination. So the problem is I want to go from S to t and I am interested in taking the shortest path.

Consider the graph above, there is vertex marked S and another vertex marked t . Now how many paths (different) are there between S and t ? We see there are 5 different paths from S to t .

The 5 different paths are as follows.



(3)

Since we are interested in shortest path, we would take 2nd path which is of length 7.7 as it is the shortest way to get to the destination.

So now the obvious question is can we always list out all the paths in the graph and take whichever is the minimum. (shortest)

The answer is that this is not feasible. Why because there may be huge no of paths between s and t which takes huge amount of time to enumerate and compute their lengths.

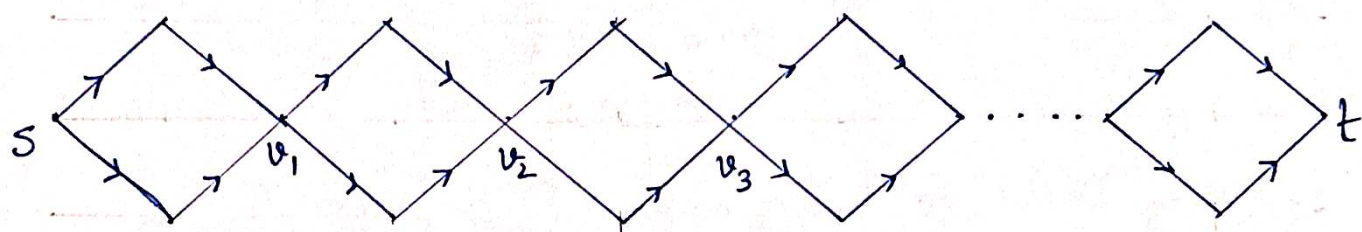
There may be a graph with n vertices and there are more than n paths. Then it is simple matter. just list out all the n paths, compute the lengths of the paths and take the shortest one.

Another important point to consider is that if the graph has a cycle. Will you go along a cycle? You are looking for a shortest paths so obviously you would not want a cycle. In that case you ignore the cycle while listing out all the paths.

(4)

So we see that listing out all the paths in a graph is not a feasible thing and the reason for this is that the number of paths from s to t can be very large. What do I mean by ~~very~~ very large?

Let us take an example



Consider the above graph, edges are all directed left to right. Let us try to find out the no of path in such a graph with n vertices. We see in the graphs there are many diamonds.

How many such diamonds are there? or let's say do it other way round.

If there are k such diamonds then how many vertices are there? k diamonds would count as $3k+1$ vertices. ex, 1 diamond $3 \times 1 + 1 = 4$ vertices
2 diamonds $3 \times 2 + 1 = 7$ vertices. etc. So if there are k diamonds, there will be $3k+1$ vertices.

Now how many paths are there between s and t ?

It is to be observed that for each diamond, I can take the top part or the bottom part, so there are two options in each diamond and this gives us 2^k different paths between s and t . Now what is the relation between n and k ?

$$3k+1 = n$$

$$k \approx n/3.$$

So k is roughly $n/3$ and so the no of different paths in terms of n is roughly $2^{n/3}$.

Now how large is the no $2^{n/3}$?

If we have a graph with say 90 vertices in it then the no of different paths would be $2^{90/3} = 2^{30}$.

How large is 2^{30} ? This is the total no of particles in the universe and this quantity is achieved in a graph with 90 vertices only.

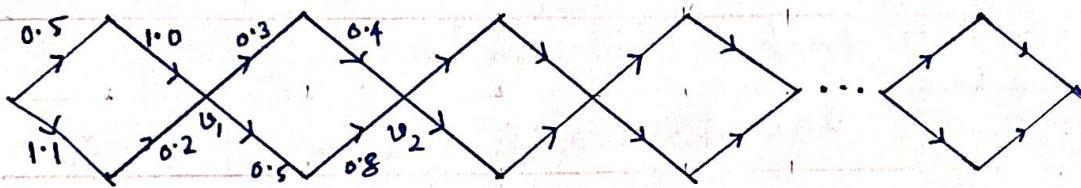
In reality the shortest paths are computed in a huge graphs having say million of vertices.

So you see it does not make sense to list out all the paths and find the shortest

e.

What would we do now? How can we solve this problem? Can you think of a way of solving this particular example before finding the general solution of this problem

Suppose following are the lengths associated with each edge in the example



Now what is it that you can do in this example to figure out the shortest path between s and t .

Let us look at the first diamond. To reach v_1 from s there are two options one is of length 1.5^(top) and the other is of length 1.3 (the bottom one). So we see that if we take the bottom path I can reach v_1 with length 1.3. From v_1 to reach v_2 we again have two options via top the length is 0.7 and via bottom the length is 1.3. So we will take the upper one and reach v_2 with total length of 2.0.

So what we essentially see is that to reach the destination, I will first have to reach v_1 , then I will have to reach v_2 , then to v_3 and so on.

(7)

So, first I will have to go to v , from s , and while going from s to v , I can choose the better option of going from s to v_1 . And we have to go to from v_1 to v_2 as there are no other possibility in this graph. and we will again choose the better option of going.

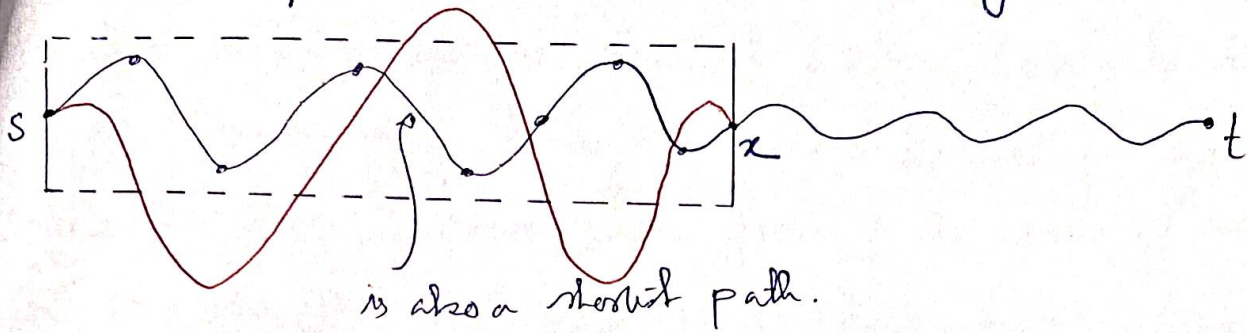
This strategy works in the graph but it may not always work. In general I have to get to vertex t starting from s . but I don't know which is the 1st unless I should get to from s . There is no such clear demarcation like to go from s to t , I have to go via this and this vertex. we can not say such a thing. So I can not say If I have to go from s to t , I can not say which is the 1st unless I have to get to. So this the problem.

How will we do this? one important properties of the shortest path is going to help us.

The property is this.

Suppose there is a shortest path from s to t , ~~as shown~~ and x is an immediate vertex on the path. then the path from s to x is also the shortest path.

Let us explain this with the following figure.



Now why the part of the path from s to x should also be a shortest path?

We will prove it by contradiction.

Suppose there were better path between s and x , then the claim is that the shortest path between s and t is this ~~red~~ path from s to x followed by the blue path from x to t . It is also to be noted that there is nothing particular about the vertex x . This claim is true for any vertex on this path.

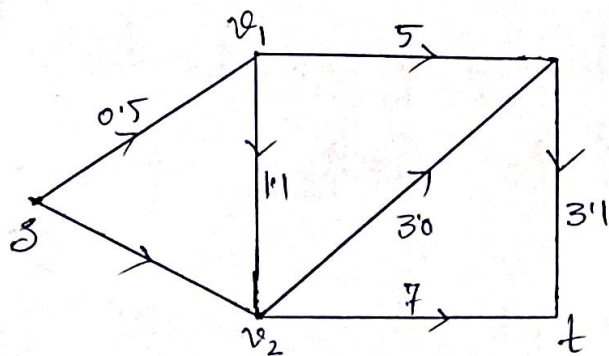
It is also to be noted that the red path may be of equal length to the blue path from s to x but never less than that. So the red path is also the shortest path.

So this gives you the following idea: If you find how to find the shortest path from s to t , we actually end up having to computing

the shortest path from s to many other vertices actually all the vertices on the ~~shortest~~ path. So what the algorithm we are discussing today is ~~actul~~ actually going to compute the shortest path from s to every vertex in the graph. not just t , in fact ~~forget~~ what we are going to do is forget t . and just compute the shortest path from s to every other vertex in G . Once we compute the shortest path from s to every vertex in G we also get the shortest path from s to t .

The problem is called single source shortest path because we are starting from one particular source and from that source we are computing the shortest path to every other vertex in G . Sometime this problem is abbreviated SSSP. standing for single source shortest path.

Now given that we are not just interested in the shortest path from s to t but the shortest path from s to every vertex in the graph, ~~as~~ let's see what we can say about the example graph that we have considered earlier.

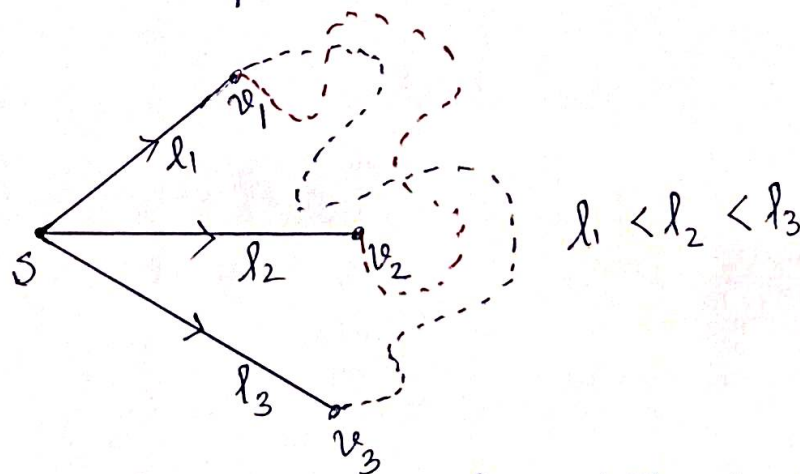


Just looking at the figure ~~not~~ and not using your head too much which vertex from s looks immediate to be in the shortest path? Obviously the adjacent vertices are the candidate. The one whose distance is minimum would be chosen.

Looking at the figure, the length of the shortest path from s to v_1 is 0.5. But if you look at the shortest path from s to v_2 , it is not 2.0, it is ~~0.5 + 1.1~~ $0.5 + 1.1 = 1.6$ via v_1 .

So we can not say the length of the shortest path from s to every adjacent vertex right away.

Let us explain this with the following figure



Suppose there are three adjacent vertices to s namely v_1 , v_2 and v_3 . Their lengths are l_1 , l_2 & l_3

Also without loss of generality assume that $l_1 < l_2 \leq l_3$. Then I know for a fact that the shortest path from s to v_1 is l_1 .

The reason for this is that any other path has to take an edge out of s and the length of that path would be at least the length of that edge. For example as there are three edges out of s with length l_1 , l_2 and l_3 , it can take any of the edge. If it takes l_1 edge, length would be at least l_1 , If it takes l_2 edge length would be at least l_2 and finally if it takes l_3 edge the length would be at least l_3 . as edge lengths are all positive. In this example the shortest path from s to v_1 would be l_1 because if we allow the path via v_2 and somehow reach v_1 , the ~~length~~ total length of the path has to be at least l_2 because the length of the path (shown in red) from v_2 to v_1 is only non-negative.

Similarly if we allow a path via v_3 , ~~that~~ and somehow reach v_1 , the total length of the path has to be at least l_3 .

So, what do understand from the above explanation is this. following.

the shortest path from s to v_1 is l_1 ($l_1 < l_2 < l_3$).

Let us use d to denote distance of a vertex from s .

So we ~~now~~ now know $d(v_1) = l_1$

what can we say about distance of vertex v_2 ?

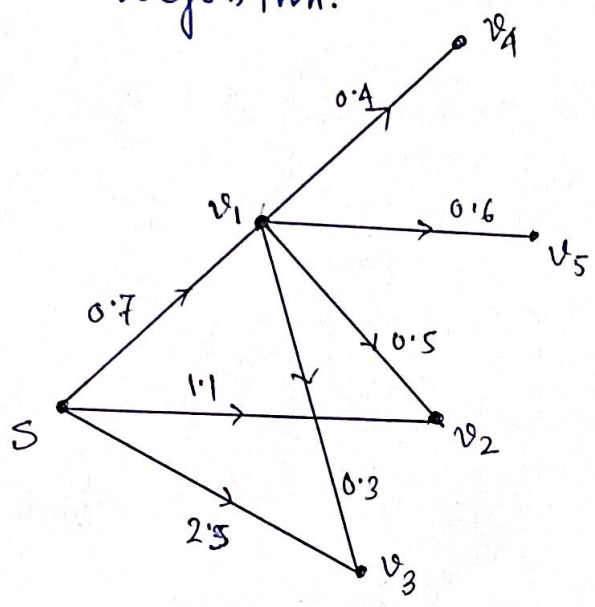
It is greater than l_1 and less than or equal to l_2
i.e. $l_1 < d(v_2) \leq l_2$

Similarly distance of v_3 is less than or equal to l_3 i.e.
 $l_1 < d(v_3) \leq l_3$.

So to start with if s is the source vertex and v_1, v_2 , and v_3 are the three adjacent vertices, then we know the exact distance of one of the vertices. we do not know the correct distance of all the other vertices but we have an upper bound on the actual distance in a sense the actual distance is only less than these values (i.e. less than these upper bound)

Well now we know the actual distance of v_1 . Now how can we extend this idea to find out distance from s to every vertex
we have just taken an example and soon we will

criticize the algorithm.



$$\begin{aligned} d(v_1) &= \boxed{0.7} \\ d(v_2) &\leq 1.1 \\ d(v_3) &\leq 2.3 \end{aligned}$$

$$d(v_4) \leq 1.1$$

$$d(v_5) \leq 1.3$$

From s we know the correct distance to v_1 . Now we need to find out the correct distance of some other vertices.

Suppose there are four more edges going out of v_1 as shown in the figure. Instead of having lengths l_1, l_2 & l_3 ~~lets~~ let us have the numbers as shown.

we know the correct distance of v_1 which is 0.7 let us encircle it. There may be other edges and vertices between these vertices but I am ~~now~~ looking at only the edges going out of v_1 and we see that there are ~~any~~ ^{only} four edges going out of it.

Now looking at the edges can I say for certain, what the shortest path to some other vertex is? I am not saying any specific vertex.

(14)

I just want to know correct shortest path to some other vertex. Let us look at what the idea is. Let us look at vertex v_3 . Till now we had only seen only the edge (s, v_1) and we had said the distance of v_3 from s is less than 2.3 . Now Given that we know the correct distance v_1 from s can we say something better about the distance of v_3 from s ?

Because now we know the edge (v_1, v_3) . I know the length of the shortest path from s to v_1 , which is 0.7 , I see the length of the edge (v_1, v_3) which is 0.3 . So the new length of the path from s to v_3 via v_1 is $0.7 + 0.3 = 1$. So we have found another path from s to v_3 of length 1 . So we can say the distance from s of v_3 from s is ≤ 1 i.e. $d(v_3) \leq 1$. That's all we can say about this part.

Let us look at vertex v_2 .

What can I say about the distance of v_2 from s ?

Well I have found another path to v_2 , the path which goes from s to v_1 and then follows the edge (v_1, v_2) .

But this path, any such path for that matter is going

we a length of atleast 1.2. In particular there is only one such path at this point and has a length 1.2. Since the earlier path had a length of 1.1., I have not found a better path to v_2 .

What can we say about vertex v_4 ?

Well there was no path from s to v_4 but now we see a path from s to v_4 via v_1 . ~~whose length is~~ First we have to get to v_1 . I know the shortest path from s to v_1 ~~the~~ which is of length 0.7. and there is an edge ~~from~~ v_1 to v_4 of length 0.4. So $d(v_4) = 1.1$ and similarly distance to v_5 is 1.3 ($= 0.7 + 0.6$) and these are the new upperbound in the distance we call them tentative distances. The distance of v_1 is ofcourse fixed $d(v_1) = 0.7$ and it is final. This distance will not change I mean will not be reduce but the other distance might reduce as the algorithm proceeds.

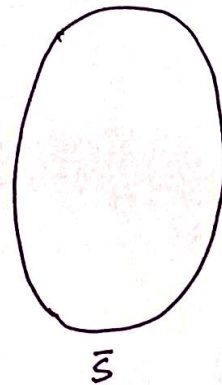
For example the distance to the vertex v_3 . The tentative distance which was 2.3 earlier, now reduced to 1.0. Distance to v_2 did not reduce. Distance to v_4 , earlier it was not reachable at all (lets say it was infinite earlier) and now it has come down from

finite to 1.1.

Now what I am claiming is that at all these paths from v_1 and smallest of them is the correct distance. In this example smallest distance at this point ~~from v_1~~ is the distance to v_3 which is 1. and this is the correct distance shortest distance from ~~v_1~~ of v_3 from s .

So let us first see what the algorithm is and we will have to prove its correctness and then we will look at its implementation. This is not a straight forward algorithm so let us understand the algorithm first.

The algorithm is going to have two set of vertices S and $\bar{S}$. ~~what does as follows~~



$\forall v \in S$ $d(v)$ is the shortest path from s

$\forall v \in \bar{S}$ $d(v)$ is an upper bound on the length of the path from s to v .

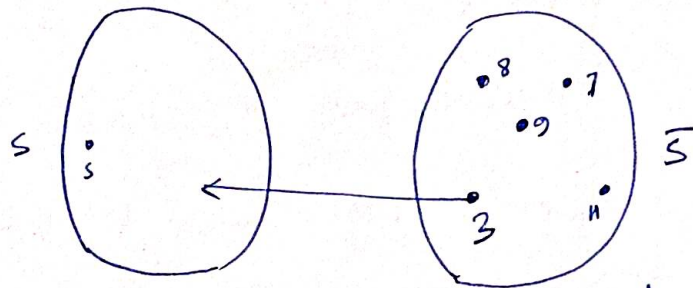
the set S denote the set of vertices for which we have found the shortest distance from the source vertex s . So what is going to happen in every step of the algorithm is that we would find ~~a~~ the shortest path to one more vertex. When I say I found the shortest path, I mean the 'shortest path' not an upper bound or any such thing. I have found the correct value of the shortest path from s to that vertex. So at every step I include one vertex into the set S . So S contains those vertices for which I have already found the correct distance of the shortest path from s to that vertex and $\bar{S}$ contains the vertices for which ~~I~~ we have not yet found the shortest path. We just have an upper bound on the value of the shortest path for this set of vertices. We know the shortest path is less than these ~~values~~ numbers. But I do not really know, what is the correct value of the shortest path. So each vertex in $\bar{S}$ has an upper bound. The upper bound is a quantity which is only larger than the shortest path. It can never be shorter than the shortest path.

So, the invariant that ~~the~~ algorithm is going to maintain is the following

for $\forall v \in \bar{S}$ $d(v)$ is the upper bound &
 $\forall v \in S$ $d(v)$ is the actual value.

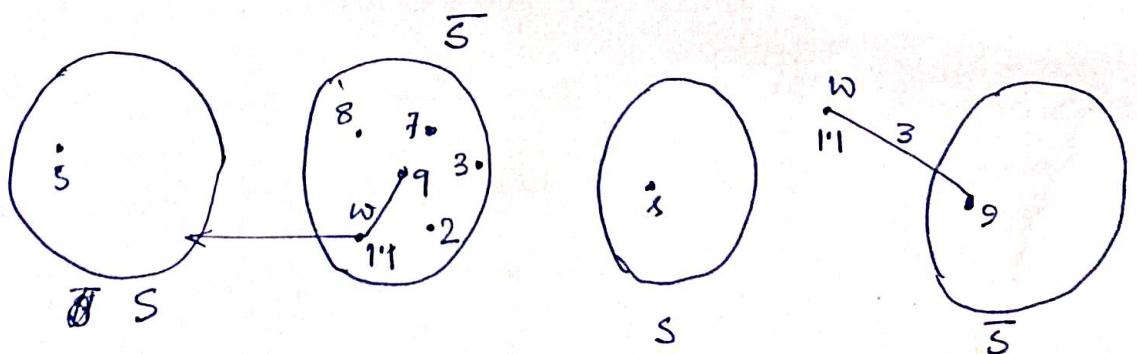
Now we will tell you how do we figure the vertex ~~no~~ vertex which have to be moved from $\bar{S}$ to S . and what happens when we move that vertex from $\bar{S}$ to S .

The vertex to be moved from $\bar{S}$ to S is the vertex which has min $d()$ value. For example



Suppose that there are vertices in $\bar{S}$ with tentative $d()$ as 7, 9, 11, 8, 3. The vertex with $d(v) = 3$ is moved from $\bar{S}$ to S . Now what happens when this movement is done? How does $d(v)$ values change in $\bar{S}$. i.e. how do we update the $d(v)$ values in $\bar{S}$. That is what we have to determine now.

Suppose the vertex ~~is~~ is w which we are now moving.



On the left side we have S and $\bar{S}$ on the right side we have $\bar{S}-w$. Suppose there is an edge from

• a vertex in $\bar{S}-w$ whose $d()$ value is '9' and suppose length of that edge from w to that vertex is 3. So what is the length of the shortest path to w ? When we move w from $\bar{S}$, the label of w remains same as it is the minimum. The right distance from s to the vertex w will be 4.1. We know that from s we can reach using the path of length 4.1 which means that we can reach the vertex with label 9 by using the path of length $4.1 + 3 = 7.1$. So 9 can now change the label of this vertex from 9 to 4.1 and this is the new upper bound or length of the shortest path from s to this vertex. So this is how we are going to update these labels. But we have not discussed why it is correct, we will do the correctness proof later.

Let us now write down the pseudocode of the algorithm. Initially the distance labels of all the vertices would be ∞ . If you recall this are the upper bound of the distance length of the shortest path from s to that vertex. Since initially we do not have any clue of what these labels be, we give every vertex a distance label of $d(v) = \infty$ and $d(s) = 0$.

Now at any step, I should take the vertex with the smallest distance labels. All these vertices ~~are~~ should be stored into a ^{min} heap based on their distance labels. ~~The min decrease~~ using them we decrease the priority of the start vertex by $H.\text{dec_prio}(S, 0)$. As a result the root/top element of the min heap will be s . The pseudo code of the algorithm is given below.

```

 $\forall v \in V \quad d(v) = \infty, H.\text{insert}(v, d(v))$ 
 $d(s) = 0. \quad H.\text{dec\_prio}(s, 0)$ 
while !  $H.\text{empty}()$  do
{
     $v = H.\text{delete\_min}()$ 
     $\forall w$  adjacent to  $v$  do
         $\hat{d}(w) = \min(d(w), d(v) + l(v, w))$ 
         $H.\text{dec\_prio}(w, d(w))$ 
}

```

Let us now explain the algorithm. • Initially the labels of every vertex is set to ∞ . The label of start vertex s is set to zero i.e. $d(s) = 0$ by the call $H.\text{dec_prio}(s, 0)$.

Now at any step we should pick the vertex with the smallest distance labels. from the heap which contains distance labels of all the vertices.

In the heap initially the top element will be the start vertex s . The generic step of the algorithm would be to remove the min vertex with its min label by $v = H.deleteMin()$ operation. So we get the min vertex as v . Now we should look at edges going out of this vertex v . i.e. we are looking at the adjacent & adjacent vertices of v . So for all ~~vertices~~ w which are adjacent to v we need to update their distance label. All these w already has some label and these labels are an upper bound on the shortest path. from s . But as a result of moving v from S to s the distance label of ~~the~~ all w s are to be updated. if we have a better path to w via v . The and the label of the path be

$$d(w) = \min(d(w), d(v) + wv)$$

So this is the other path we are seeing to w if this is shorter than the earlier label. It is to be noted that when we remove a vertex from S to s that vertex should be removed from the heap also and the deleteMin() operation exactly does that. Now once we ~~update~~ have

reduced labels of the and adjacent vertices of u we decrease their ~~priority~~ label in the Heap by the operation $\text{H-decreprio}(w, d(w))$ $\text{H-decreprio}(w, d(w))$.

It is to be noted here that this $\text{H-decreprio}(w, d(w))$ may not be called for all w , it will be called only for those w for which the distance labels are updated. ~~It may~~ You can implement the algorithm accordingly. ~~for, if $d(w)$ decreases should you call the H-decreprio call that.~~

So what we have found out ~~to~~ at the end of the algorithm? we have found out only the length of the shortest path for s to every other vertices ~~is~~ and they are sitting in $d()$'s.

Now what is the running time of the algorithm?

Inside while loop we are spending time proportional to the and degree of each vertex.